Polytech Lyon

Optimisation Discrète

Projet : Vehicle Routing Problem with Time Windows

Rapport de projet

Étudiants :	Nom Prénom
	Nom Prénom
Enseignant :	Stéphane Bonnevay
Date :	20 mars 2026

Ce rapport présente notre travail sur la résolution du problème de tournées de véhicules avec fenêtres de temps (VRPTW) à l'aide de métaheuristiques. Le sujet demande de modéliser le problème, de structurer le code, de déterminer le nombre minimal de véhicules, de générer des solutions aléatoires, puis d'implémenter et comparer deux métaheuristiques.

Table des matières

1 Introduction **4**

2 Organisation du projet **4**

2.1 Phase 1 : Fondations du projet 4

2.2 Phase 2 : Détermination du nombre minimal de véhicules 4

2.3 Phase 3 : Génération de solutions initiales 4

2.4 Phase 4 : Première métaheuristique 4

2.5 Phase 5 : Deuxième métaheuristique 5

2.6 Phase 6 : Protocole expérimental et comparaison 5

3 Modélisation du problème **5**

3.1 Description générale 5

3.2 Ensembles 5

3.3 Paramètres 5

3.4 Variables de décision 6

3.5 Fonction objectif 6

3.6 Contraintes principales 6

3.6.1 Visite unique de chaque client 6

3.6.2 Conservation du flot 6

3.6.3 Capacité 6

3.6.4 Fenêtres de temps 6

3.7 Remarque sur l'utilisation d'OR-Tools 7

4 Structure du code **7**

4.1 Évolution de l'architecture 7

4.2 Justification des choix de fichiers 7

4.3 Pourquoi cette structure nous a semblé adaptée 8

4.4 Lien entre structure logicielle et démarche scientifique 8

4.5 `main.py` 9

4.6 `model.py` 9

4.7 `io_utils.py` 9

4.8 `evaluate.py` 9

4.9 `solver.py` 9

4.10 `neighbors.py` 9

4.11 `meta/` 10

5 Détermination du nombre minimal de véhicules **10**
5.1 Principe général . . . 10
5.2 Borne inférieure théorique . . . 10
5.3 Choix méthodologique . . . 11
5.4 Heuristique utilisée pour tester une valeur de k . . . 11
5.5 Critère de faisabilité . . . 11
5.6 Remarque . . . 11
5.7 Résultats obtenus sans fenêtres de temps . . . 12
5.8 Influence des fenêtres de temps sur la borne inférieure . . . 12
5.9 Première intégration effective des fenêtres de temps . . . 13
5.10 Premiers résultats VRPTW . . . 13
5.11 Limites observées à ce stade . . . 13
5.12 Première extension des métaheuristiques au cas VRPTW . . . 14
5.13 Premier essai expérimental VRPTW . . . 14
5.14 Campagne VRPTW en mode rapide . . . 14
5.15 Analyse de cette première campagne VRPTW . . . 15
5.16 Campagne VRPTW en mode long . . . 15
5.17 Analyse de la campagne VRPTW longue . . . 15

6 Génération d'une solution initiale **16**
6.1 Choix retenu . . . 16
6.2 Cas sans fenêtres de temps . . . 16
6.3 Cas avec fenêtres de temps . . . 16
6.4 Intérêt de cette construction . . . 17
6.5 Rôle de cette solution . . . 17

7 Choix des méthodes de résolution **17**
7.1 Voisinages envisagés . . . 17
7.2 Première implémentation des voisinages . . . 17
7.3 Première implémentation du recuit simulé . . . 18
7.4 Résultats préliminaires . . . 18
7.5 Première implémentation de la recherche tabou . . . 18
7.6 Observation importante sur le coût de calcul . . . 18
7.7 Résultats préliminaires de la recherche tabou . . . 19
7.8 Mise en place du protocole expérimental . . . 19
7.9 Premiers résultats comparatifs . . . 19
7.10 Premier essai en mode long . . . 20
7.11 Extension du test long à `data102.vrp` . . . 20

7.12 Résultats sur `data1101.vrp` et `data1102.vrp` 21
7.13 Résultats sur `data111.vrp` et `data112.vrp` 21
7.14 Résultats sur `data1201.vrp` et `data1202.vrp` 21
7.15 Résultats sur `data201.vrp` et `data202.vrp` 22
7.16 Synthèse de la première campagne longue 22

8 Format des données **22**

9 Bonus : résolution exacte sur sous-instances **23**

10 Limites et perspectives **24**

11 Conclusion **24**

1. Introduction

Le projet étudié porte sur le **Vehicle Routing Problem with Time Windows (VRPTW)**. L'objectif est de déterminer un ensemble de tournées partant et revenant au dépôt, de manière à desservir chaque client exactement une fois, en respectant les contraintes du problème et en minimisant la distance totale parcourue.

Le sujet demande explicitement :

- de modéliser le problème et de structurer le code ;
- de déterminer, pour chaque jeu de données, le nombre minimum de véhicules ;
- de créer un générateur aléatoire de solutions ;
- d'implémenter deux métaheuristiques ;
- de comparer les résultats obtenus ;
- de commencer sans tenir compte des fenêtres de temps, puis de les ajouter ensuite.

Dans un premier temps, nous considérons donc le problème **sans fenêtres temporelles**, ce qui revient à traiter un **Capacitated Vehicle Routing Problem (CVRP)**. Les fenêtres de temps seront prises en compte dans une seconde phase.

2. Organisation du projet

Afin de répondre progressivement aux différentes questions du sujet, nous avons découpé le travail en plusieurs phases.

2.1. Phase 1 : Fondations du projet

Cette phase regroupe :

- la modélisation mathématique du problème ;
- la définition de la structure du code.

2.2. Phase 2 : Détermination du nombre minimal de véhicules

Cette phase est décomposée en deux sous-parties :

- sans fenêtres de temps ;
- avec fenêtres de temps.

2.3. Phase 3 : Génération de solutions initiales

Le sujet demandant un générateur aléatoire, nous avons choisi de construire des **solutions aléatoires faisables**.

2.4. Phase 4 : Première métaheuristique

Nous avons retenu le recuit simulé comme première métaheuristique. Une première version fonctionnelle a été implémentée afin de valider la chaîne complète :

- génération d'une solution initiale ;
- génération d'un voisin faisable ;
- acceptation déterministe ou probabiliste ;
- mémorisation de la meilleure solution rencontrée.

2.5. Phase 5 : Deuxième métaheuristique

Nous avons retenu la recherche tabou comme seconde métaheuristique. Une première version a été mise en place afin de disposer rapidement des deux approches demandées par le sujet avant de lancer les comparaisons expérimentales détaillées.

2.6. Phase 6 : Protocole expérimental et comparaison

Une première infrastructure expérimentale a maintenant été mise en place afin de distinguer clairement deux usages :

- un mode « rapide » destiné à valider les implémentations et à obtenir un premier ordre de grandeur ;
- un mode « long » destiné aux comparaisons plus rigoureuses, avec davantage d'itérations et plusieurs répétitions.

3. Modélisation du problème

3.1. Description générale

On considère un dépôt unique et un ensemble de clients. Chaque client possède :

- des coordonnées cartésiennes ;
- une demande ;
- éventuellement une fenêtre de temps ;
- un temps de service.

Les véhicules sont supposés identiques et possèdent tous la même capacité maximale Q.

L'objectif principal est de minimiser la distance totale parcourue. Le nombre de véhicules n'étant pas fixé dans le sujet, il doit être déterminé.

3.2. Ensembles

$$V = \{0, 1, \ldots, n\}$$

désigne l'ensemble des sommets, où :

- 0 représente le dépôt ;
- $C = \{1, \ldots, n\}$ représente l'ensemble des clients.

3.3. Paramètres

- d_{ij} : distance entre les sommets i et j ;
- q_i : demande du client i ;
- Q : capacité maximale d'un véhicule ;
- a_i : date de début de la fenêtre de temps du client i ;
- b_i : date de fin de la fenêtre de temps du client i ;
- s_i : durée de service chez le client i.

3.4. Variables de décision

Une modélisation classique consiste à utiliser des variables binaires :

$$x_{ij} = \begin{cases} 1 & \text{si un véhicule va directement de } i \text{ vers } j, \\ 0 & \text{sinon.} \end{cases}$$

Pour la version avec fenêtres de temps, on introduit également :

$$t_i = \text{instant de début de service au client } i.$$

3.5. Fonction objectif

L'objectif est de minimiser la distance totale parcourue :

$$\min \sum_{i \in V} \sum_{j \in V,\, j \neq i} d_{ij} x_{ij}.$$

3.6. Contraintes principales

3.6.1 Visite unique de chaque client

Chaque client doit être visité exactement une fois :

$$\sum_{j \in V,\, j \neq i} x_{ij} = 1 \qquad \forall i \in C$$

$$\sum_{i \in V,\, i \neq j} x_{ij} = 1 \qquad \forall j \in C$$

3.6.2 Conservation du flot

Si un véhicule arrive chez un client, il doit également en repartir :

$$\sum_{j \in V} x_{ij} = \sum_{j \in V} x_{ji} \qquad \forall i \in C$$

3.6.3 Capacité

Pour chaque véhicule, la somme des demandes des clients affectés à sa tournée ne doit pas dépasser Q.

Cette contrainte sera traitée dans notre implémentation par le calcul explicite de la charge de chaque tournée.

3.6.4 Fenêtres de temps

Dans la seconde partie du projet, on ajoute :

$$a_i \leq t_i \leq b_i$$

et, pour tout déplacement de i vers j,

$$t_j \geq t_i + s_i + d_{ij}.$$

3.7. Remarque sur l'utilisation d'OR-Tools

Nous avons choisi d'utiliser OR-Tools uniquement pour des **tests sur petites instances** et pour la validation de certaines hypothèses. En effet, sur des instances plus grandes, l'utilisation directe d'un solveur exact peut devenir trop coûteuse en temps de calcul. La résolution principale du projet repose donc sur des méthodes heuristiques et métaheuristiques.

4. Structure du code

La structure du code a été pensée de manière modulaire, afin de séparer clairement les rôles de chaque composant.

```
src/
  main.py
  model.py
  io_utils.py
  evaluate.py
  solver.py
  neighbors.py
  meta/
    simulated_annealing.py
    tabu_search.py
```

4.1. Évolution de l'architecture

La structure actuelle n'a pas été figée dès le début du projet. Elle est le résultat d'une évolution progressive, guidée à la fois par le sujet et par les besoins apparus au cours de l'implémentation.

Dans une première étape, nous avions surtout besoin de lire correctement les instances et de manipuler les données du problème. Cela a conduit à isoler très tôt un module de lecture, afin de ne pas mélanger le parsing des fichiers `.vrp` avec la logique algorithmique.

Ensuite, à mesure que plusieurs méthodes de construction et d'amélioration ont été ajoutées, il est devenu clair qu'il fallait éviter de dupliquer partout les mêmes calculs de distance, de charge ou de faisabilité. C'est cette réflexion qui a motivé la centralisation de ces calculs dans `evaluate.py`. Ce choix a été important, car il garantit que les heuristiques, les voisinages et les métaheuristiques reposent tous sur la même définition d'une solution faisable.

Dans un troisième temps, lorsque le projet a commencé à inclure des opérateurs de voisinage puis deux métaheuristiques différentes, nous avons séparé plus explicitement :

- la construction des solutions ;
- l'évaluation des solutions ;
- la génération des voisins ;
- la logique propre à chaque métaheuristique ;
- le pilotage global des expériences.

Cette évolution n'est donc pas purement esthétique. Elle répond à un besoin de lisibilité, de réutilisation du code et de réduction des erreurs.

4.2. Justification des choix de fichiers

Le découpage en fichiers a été pensé pour suivre les grandes responsabilités du projet.

- `io_utils.py` regroupe tout ce qui concerne la lecture des données. Nous avons préféré isoler cette partie, car elle dépend fortement du format des fichiers et ne doit pas polluer la logique des algorithmes.
- `evaluate.py` joue le rôle de noyau commun. Nous avons choisi d'y placer tous les calculs de distance, de charge et de faisabilité, car ces éléments sont utilisés par presque tous les autres modules.
- `construction.py` contient les méthodes de construction de solutions. Nous avons fait ce choix pour distinguer clairement les solutions initiales des méthodes d'amélioration.
- `neighbors.py` rassemble les transformations locales des solutions. Il nous a semblé plus propre d'avoir un module dédié aux mouvements, plutôt que de disperser ces opérations dans chaque métaheuristique.
- le dossier `meta/` contient les métaheuristiques elles-mêmes. Cette séparation facilite la comparaison des approches, car chaque méthode dispose de son propre fichier, avec ses paramètres et sa logique.
- `solver.py` a été introduit plus tard, lorsque le besoin d'organiser les expériences est apparu. Nous avons choisi de ne pas surcharger `main.py` avec toute la logique des campagnes de tests.
- `main.py` reste volontairement léger : son rôle est de lancer les traitements et d'afficher des synthèses, pas de contenir toute l'intelligence du projet.

4.3. Pourquoi cette structure nous a semblé adaptée

Nous n'avons pas cherché à construire une architecture complexe dès le départ. Au contraire, nous avons essayé de garder une structure simple, puis de la faire évoluer uniquement lorsque cela devenait utile.

Par exemple, tant que le projet ne contenait qu'une heuristique gloutonne, il n'était pas indispensable de distinguer fortement tous les rôles. En revanche, dès que plusieurs algorithmes ont commencé à partager les mêmes briques, le découpage modulaire est devenu nécessaire.

Cette manière de faire nous a paru préférable à une structure trop ambitieuse dès le début. Elle nous a permis :

- d'avancer progressivement sans sur-ingénierie ;
- de refactoriser seulement quand un besoin concret apparaissait ;
- de garder un code plus facile à tester, à modifier et à documenter ;
- de faire correspondre plus naturellement chaque fichier à une idée claire du projet.

4.4. Lien entre structure logicielle et démarche scientifique

Enfin, cette structure a aussi été pensée en fonction du rapport expérimental demandé par le sujet. En séparant clairement les responsabilités, il devient plus facile d'expliquer :

- comment une solution est construite ;
- comment elle est évaluée ;
- comment elle est modifiée localement ;
- comment les métaheuristiques exploitent ces briques ;
- comment les expériences sont lancées et comparées.

Autrement dit, l'organisation du code soutient directement la clarté de l'analyse présentée dans le rapport.

4.5. `main.py`

Point d'entrée du programme. Il permet de charger une instance, de lancer l'algorithme choisi et d'afficher ou sauvegarder les résultats.

4.6. `model.py`

Ce fichier contient les structures de données principales :

- client ;
- dépôt ;
- instance ;
- solution.

4.7. `io_utils.py`

Ce module est chargé de la lecture des fichiers `.vrp` et de la sauvegarde des solutions au format JSON.

4.8. `evaluate.py`

Ce fichier regroupe les fonctions de calcul :

- distance totale ;
- faisabilité d'une tournée ;
- respect de la capacité ;
- respect des fenêtres de temps.

Dans l'état actuel du projet, ce module joue un rôle central car il sert de base commune à toutes les étapes suivantes. Nous y avons regroupé les calculs de distance d'une tournée, de charge, ainsi que l'évaluation temporelle détaillée d'une route.

Plus précisément, pour une tournée donnée, nous calculons successivement :

- le temps d'arrivée à chaque client ;
- le temps d'attente éventuel si le véhicule arrive avant l'ouverture de la fenêtre ;
- l'instant de début de service ;
- l'instant de départ après service ;
- le temps de retour au dépôt.

Cette évaluation permet de tester séparément la faisabilité en capacité et la faisabilité temporelle. Elle sera réutilisée directement pour le générateur aléatoire de solutions initiales, puis pour les opérateurs de voisinage et les métaheuristiques.

4.9. `solver.py`

Interface de haut niveau permettant de lancer les différentes méthodes de résolution.

4.10. `neighbors.py`

Ce module contiendra les opérateurs de voisinage utilisés par les métaheuristiques, par exemple :

- déplacement d'un client ;

— échange de clients ;
— inversion de segment (`2-opt`) ;
— opérateurs intra-route et inter-route.

Dans l'état actuel du projet, une première version de ce module a été implémentée. Elle permet de générer des voisins faisables à partir de trois opérateurs classiques :

— `relocate`, qui déplace un client d'une position à une autre, éventuellement dans une autre tournée ;
— `swap`, qui échange deux clients ;
— `2-opt`, qui inverse un segment à l'intérieur d'une tournée.

Chaque voisin généré est filtré par le module `evaluate.py`, ce qui garantit que seuls des candidats faisables sont conservés. Cette décision simplifie la mise en place des métaheuristiques, qui pourront se concentrer sur la stratégie d'exploration plutôt que sur la gestion locale des contraintes.

4.11. `meta/`

Ce dossier contiendra les deux métaheuristiques retenues dans le projet :

— recuit simulé ;
— recherche tabou.

5. Détermination du nombre minimal de véhicules

5.1. Principe général

Le sujet demande de déterminer, pour chaque jeu de données, le nombre minimum de véhicules à utiliser.

Nous avons choisi de traiter cette question en deux temps :

— d'abord sans fenêtres de temps ;
— ensuite avec fenêtres de temps.

Dans cette première étape, nous ne considérons donc que :

— la capacité des véhicules ;
— la nécessité de construire de vraies tournées partant et revenant au dépôt ;
— la distance entre les clients et le dépôt.

Autrement dit, nous traitons d'abord une version de type **CVRP** avant d'ajouter les fenêtres de temps.

5.2. Borne inférieure théorique

Une borne inférieure immédiate du nombre de véhicules est obtenue à partir de la somme des demandes :

$$LB = \left\lceil \frac{\sum_{i \in C} q_i}{Q} \right\rceil .$$

Cette borne est nécessaire mais pas toujours suffisante, car elle ne tient compte que de la capacité totale disponible. Elle ignore :

— la répartition des demandes entre les tournées ;
— la structure géographique des clients ;

— la cohérence réelle des trajets.

Deux clients compatibles en termes de charge peuvent en effet être très éloignés, ce qui rend leur regroupement peu pertinent dans une même tournée.

5.3. Choix méthodologique

Nous avons décidé de ne pas déterminer le nombre minimal de véhicules uniquement à partir des demandes, comme dans un simple problème de remplissage. En effet, même sans fenêtres temporelles, il est nécessaire de tenir compte de la distance afin de construire des tournées réalistes.

Notre méthode consiste donc à :

1. calculer la borne inférieure LB ;
2. fixer $k = LB$;
3. essayer de construire une solution faisable avec exactement k véhicules ;
4. si aucun ensemble de tournées faisables n'est obtenu, incrémenter k ;
5. retenir le premier k pour lequel tous les clients sont servis.

5.4. Heuristique utilisée pour tester une valeur de k

Pour tester si k véhicules suffisent, nous utilisons une heuristique gloutonne géographique sous contrainte de capacité.

Le principe est le suivant :

1. initialiser k tournées vides ;
2. choisir un premier client pour chaque tournée ;
3. compléter chaque tournée en ajoutant à chaque étape un client non encore servi ;
4. parmi les clients admissibles, sélectionner celui qui minimise la distance au dernier client de la tournée ;
5. ne jamais dépasser la capacité maximale Q ;
6. s'arrêter lorsque plus aucun client ne peut être ajouté à la tournée courante.

Un client est dit **admissible** si son ajout à la tournée ne fait pas dépasser la capacité du véhicule.

5.5. Critère de faisabilité

Pour une valeur donnée de k, la solution est considérée comme faisable si :

— tous les clients sont affectés à une unique tournée ;

— la charge de chaque tournée est inférieure ou égale à Q.

Dans cette phase, la distance intervient comme **critère de construction** des tournées, mais pas encore comme objectif d'optimisation final.

Du point de vue de l'implémentation, cette vérification est maintenant centralisée dans le module `evaluate.py`. Cela permet d'éviter la duplication des contrôles et garantit que les futures méthodes de construction et d'amélioration utiliseront toutes le même critère de faisabilité.

5.6. Remarque

Cette approche ne prouve pas mathématiquement que le nombre trouvé est optimal. En revanche, elle fournit une estimation cohérente du nombre minimal de véhicules, adaptée à la taille des

instances étudiées. Cette estimation sera ensuite affinée indirectement par les métaheuristiques développées dans les phases suivantes.

5.7. Résultats obtenus sans fenêtres de temps

Nous avons appliqué notre méthode de construction gloutonne géographique à l'ensemble des instances, en commençant à chaque fois par la borne inférieure

$$LB = \left\lceil \frac{\sum_{i \in C} q_i}{Q} \right\rceil.$$

Pour chaque instance testée, une solution faisable a été obtenue dès cette borne inférieure. Cela signifie que, dans le cas sans fenêtres de temps, le nombre minimal de véhicules est exactement égal à LB pour toutes les instances considérées.

Instance	Capacité	Demande totale	LB	Nombre minimal trouvé
data101.vrp	200	1458	8	8
data102.vrp	200	1458	8	8
data1101.vrp	200	1724	9	9
data1102.vrp	200	1724	9	9
data111.vrp	200	1458	8	8
data112.vrp	200	1458	8	8
data1201.vrp	1000	1724	2	2
data1202.vrp	1000	1724	2	2
data201.vrp	1000	1458	2	2
data202.vrp	1000	1458	2	2

On remarque donc que, pour ces jeux de données et sans tenir compte des fenêtres temporelles, la contrainte de capacité suffit à expliquer le nombre minimal de véhicules.

5.8. Influence des fenêtres de temps sur la borne inférieure

La borne inférieure utilisée dans notre étude est basée uniquement sur la capacité des véhicules :

$$LB = \left\lceil \frac{\sum_{i \in C} q_i}{Q} \right\rceil.$$

Cette borne ne dépend que de la demande totale et de la capacité des véhicules. Elle est donc indépendante des distances et des fenêtres temporelles.

Ainsi, l'introduction des fenêtres de temps ne modifie pas cette borne inférieure. En revanche, elle restreint fortement l'ensemble des tournées faisables. Certaines combinaisons de clients compatibles en termes de capacité peuvent devenir impossibles à réaliser lorsque leurs fenêtres de service sont incompatibles.

Par conséquent, le nombre minimal de véhicules nécessaire avec fenêtres de temps est généralement supérieur ou égal à la borne inférieure :

$$k_{TW} \geq LB.$$

L'objectif de la seconde partie de la question 2 est donc de déterminer expérimentalement le nombre minimal de véhicules lorsque les contraintes temporelles sont prises en compte.

5.9. Première intégration effective des fenêtres de temps

Après avoir d'abord travaillé sur la version capacitaire, nous avons intégré explicitement les fenêtres de temps dans la construction des tournées. Le simple prolongement de l'heuristique gloutonne précédente ne suffisait pas, car ajouter systématiquement les clients en fin de tournée se révélait trop rigide dans le cas VRPTW.

Nous avons donc fait évoluer la construction vers une logique d'insertion faisable. L'idée est de partir d'un ensemble de tournées initialement vides, puis d'insérer progressivement les clients dans les positions qui restent compatibles avec :

— la capacité du véhicule ;
— les fenêtres de temps ;
— le retour au dépôt dans la fenêtre autorisée.

Ce changement de stratégie est important du point de vue méthodologique : il illustre bien le fait qu'une structure de construction adaptée au CVRP ne se transpose pas automatiquement au VRPTW.

5.10. Premiers résultats VRPTW

Les premiers essais obtenus avec cette heuristique d'insertion montrent déjà un effet très net des fenêtres de temps sur le nombre minimal de véhicules nécessaire.

Sur un premier sous-ensemble d'instances, nous obtenons :

Instance	Borne inférieure capacité	Nombre trouvé avec fenêtres de temps
data101.vrp	8	21
data102.vrp	8	18
data1101.vrp	9	18
data1201.vrp	2	5
data201.vrp	2	5

Ces premiers résultats confirment que les fenêtres de temps modifient profondément la structure du problème. Alors que la version sans fenêtres de temps permettait souvent d'atteindre directement la borne inférieure capacité, la version VRPTW exige ici un nombre de véhicules bien plus élevé.

5.11. Limites observées à ce stade

Cette première intégration des fenêtres de temps s'accompagne toutefois d'une limite importante. Le générateur aléatoire de solutions initiales, qui fonctionne correctement dans la version sans fenêtres de temps, ne parvient pas encore à produire facilement des solutions faisables lorsqu'on active les contraintes temporelles.

En pratique, cela signifie que :

— la construction déterministe par insertion constitue actuellement notre base fiable pour la version VRPTW ;
— la partie métaheuristique avec fenêtres de temps n'est pas encore stabilisée, car elle dépend d'une initialisation faisable plus robuste.

Ce constat est utile pour la suite du projet : il montre que l'intégration des fenêtres de temps ne se limite pas à activer un test de faisabilité supplémentaire, mais demande aussi de repenser certaines briques de construction.

5.12. Première extension des métaheuristiques au cas VRPTW

Malgré cette limite, nous avons pu commencer à étendre les métaheuristiques à la version avec fenêtres de temps en utilisant une solution initiale déterministe faisable comme point de départ.

Concrètement, lorsque la génération aléatoire échoue en mode VRPTW, nous utilisons la construction par insertion faisable pour fournir une solution initiale aux métaheuristiques. Cette stratégie n'est pas encore la version finale souhaitée, mais elle permet déjà de tester le comportement des algorithmes sur le vrai problème avec contraintes temporelles.

Un premier test ciblé a ainsi été réalisé sur l'instance `data1201.vrp`. Avec $k = 5$ véhicules, le recuit simulé a pu être exécuté à partir d'une solution initiale VRPTW faisable, tout en conservant la faisabilité de la solution au cours de l'optimisation.

Cette étape constitue une transition importante : même si l'initialisation aléatoire VRPTW reste à fiabiliser, la chaîne « construction faisable + amélioration locale » devient déjà exploitable.

5.13. Premier essai expérimental VRPTW

Nous avons ensuite lancé un premier essai expérimental en mode rapide sur l'instance `data1201.vrp`, avec fenêtres de temps activées. La solution initiale faisable obtenue présente une distance de 1974.63.

Dans ce premier essai :

- le recuit simulé améliore la solution et atteint une distance de 1909.98 ;
- la recherche tabou atteint une distance de 1945.36.

Les temps de calcul observés sont d'environ 0.74 seconde pour le recuit simulé et 9.35 secondes pour la recherche tabou.

Ce résultat reste préliminaire, mais il est important : il montre que nous disposons désormais d'une première chaîne expérimentale opérationnelle sur la vraie version VRPTW, même si elle repose encore sur une initialisation déterministe plutôt que sur un générateur aléatoire pleinement robuste.

5.14. Campagne VRPTW en mode rapide

Après ce premier test ciblé, nous avons lancé une première campagne `quick` sur l'ensemble des instances disponibles, cette fois avec fenêtres de temps activées. Pour chaque instance, nous avons utilisé une solution initiale faisable, puis appliqué le recuit simulé et la recherche tabou.

Instance	k	Init.	RS	Tabou	Temps RS	Temps tabou
data101.vrp	21	2127.09	1906.31	1995.52	4.77	5.20
data102.vrp	18	1794.68	1703.70	1766.13	1.22	1.94
data1101.vrp	18	2184.86	2073.22	2079.72	2.16	2.56
data1102.vrp	16	2065.99	1926.79	1926.77	1.55	1.32
data111.vrp	14	1601.37	1601.37	1540.14	1.81	1.71
data112.vrp	12	1434.70	1392.79	1401.36	0.92	1.09
data1201.vrp	5	1748.48	1737.71	1653.86	0.83	0.60
data1202.vrp	4	2032.65	1896.22	1872.86	0.75	0.69
data201.vrp	5	1636.58	1617.42	1563.94	0.72	0.66
data202.vrp	5	1600.85	1599.76	1517.66	0.45	0.44

5.15. Analyse de cette première campagne VRPTW

Cette campagne rapide met déjà en évidence plusieurs tendances importantes.

— Les fenêtres de temps augmentent fortement le nombre de véhicules nécessaires, parfois dans des proportions très importantes par rapport à la borne inférieure capacitaire.
— Le recuit simulé améliore la solution initiale sur la majorité des instances testées, avec des gains parfois nets, comme sur `data101.vrp` ou `data202.vrp`.
— La recherche tabou améliore également les solutions, et elle surpasse même légèrement le recuit simulé sur certaines instances comme `data111.vrp`, `data1201.vrp`, `data1202.vrp` ou `data201.vrp`.
— En revanche, la recherche tabou reste plus coûteuse en temps de calcul sur l'ensemble de cette campagne rapide, avec un temps souvent compris entre 9 et 13 secondes, contre des temps généralement inférieurs à 4 secondes pour le recuit simulé.

Ces résultats sont encore préliminaires, mais ils montrent que la hiérarchie observée dans la version sans fenêtres de temps n'est pas automatiquement identique dans le cas VRPTW. La présence des contraintes temporelles modifie non seulement la faisabilité des tournées, mais aussi le comportement comparé des métaheuristiques.

5.16. Campagne VRPTW en mode long

Nous avons ensuite prolongé cette étude avec une campagne `long` complète sur les dix instances, toujours avec fenêtres de temps activées. Cette campagne reprend le même principe que la campagne rapide, mais avec davantage d'itérations et deux répétitions par instance.

Instance	k	Init. moy.	RS moy.	Tabou moy.	Temps RS	Temps tabou
data101.vrp	21	2127.09	1756.06	1852.75	20.29	49.07
data102.vrp	18	1794.68	1683.80	1610.82	4.11	13.15
data1101.vrp	18	2184.86	1851.01	1864.19	6.94	29.45
data1102.vrp	16	1913.17	1753.92	1663.26	4.46	9.47
data111.vrp	14	1601.37	1426.25	1417.19	3.09	8.46
data112.vrp	12	1434.70	1256.10	1229.63	3.69	7.14
data1201.vrp	5	2267.76	1840.63	1752.22	3.09	7.40
data1202.vrp	4	1924.75	1700.61	1633.87	2.67	5.06
data201.vrp	5	1759.27	1631.38	1449.18	2.33	5.19
data202.vrp	5	1506.60	1506.60	1282.73	1.19	2.26

5.17. Analyse de la campagne VRPTW longue

Cette campagne longue confirme et clarifie plusieurs tendances déjà visibles en mode rapide.

— Le recuit simulé améliore systématiquement les solutions initiales sur l'ensemble du corpus.
— La recherche tabou recalibrée améliore elle aussi systématiquement les solutions et devient meilleure que le recuit simulé sur une majorité d'instances VRPTW.
— Le recuit simulé garde un avantage net sur les instances les plus contraintes en nombre de véhicules, en particulier `data101.vrp`, et reste très proche de la tabou sur `data1101.vrp`.
— Sur les autres instances, la recherche tabou devient la meilleure méthode observée, avec des gains marqués sur `data102.vrp`, `data1201.vrp`, `data201.vrp` et `data202.vrp`.

— Le coût en temps de la recherche tabou augmente sensiblement sur les cas difficiles, mais reste raisonnable sur la majorité du corpus et devient acceptable au regard du gain de qualité obtenu.

Cette campagne longue modifie donc notre lecture finale du projet : dans le cas VRPTW, une recherche tabou correctement échantillonnée et recalibrée devient très compétitive, et même meilleure que le recuit simulé sur la plupart des instances testées, au prix d'un surcoût encore notable sur quelques cas difficiles.

6. Génération d'une solution initiale

Le sujet demande explicitement la création d'un générateur aléatoire de solutions. Nous avons donc choisi de construire une **solution aléatoire faisable**.

6.1. Choix retenu

Nous ne retenons pas une permutation totalement aléatoire des clients, car celle-ci produit très souvent des solutions infaisables. À la place, nous construisons la solution progressivement :

1. ouvrir une nouvelle tournée ;
2. déterminer la liste des clients admissibles dans cette tournée ;
3. choisir aléatoirement l'un de ces clients ;
4. l'ajouter à la tournée ;
5. recommencer jusqu'à ce qu'aucun autre client admissible ne puisse être ajouté ;
6. ouvrir une nouvelle tournée ;
7. répéter jusqu'à ce que tous les clients soient servis.

Dans notre implémentation actuelle, nous fixons d'abord un nombre de tournées k, en pratique le nombre minimal de véhicules obtenu dans la phase précédente pour la version sans fenêtres de temps. Nous initialisons ensuite ces k tournées puis nous ajoutons des clients un par un de manière aléatoire, sous réserve d'admissibilité.

À chaque étape, un client est considéré comme admissible si son ajout en fin de tournée conserve la faisabilité de cette tournée. Le choix final est ensuite effectué aléatoirement parmi l'ensemble des clients admissibles. Cette stratégie permet de produire des solutions variées tout en restant contrôlées.

6.2. Cas sans fenêtres de temps

Dans un premier temps, un client est admissible si son ajout ne fait pas dépasser la capacité du véhicule.

6.3. Cas avec fenêtres de temps

Dans un second temps, un client sera admissible si son ajout respecte à la fois :

— la capacité du véhicule ;
— la faisabilité temporelle de la tournée.

6.4. Intérêt de cette construction

Cette méthode présente deux avantages importants pour la suite du projet :

— elle génère plusieurs solutions initiales différentes pour une même instance, ce qui est utile pour évaluer la robustesse des métaheuristiques ;
— elle s'appuie sur le même module d'évaluation que le reste du projet, ce qui garantit une cohérence entre construction, tests de faisabilité et futurs mouvements de voisinage.

6.5. Rôle de cette solution

Cette solution initiale n'a pas vocation à être optimale. Elle sert de point de départ aux métaheuristiques.

7. Choix des méthodes de résolution

Le projet demandant l'utilisation de deux métaheuristiques, nous avons retenu les approches suivantes :

— le recuit simulé ;
— la recherche tabou.

Ces deux méthodes sont particulièrement adaptées à une résolution par voisinage du problème de tournées de véhicules.

7.1. Voisinages envisagés

Les opérateurs suivants seront étudiés :

— relocate ;
— exchange ;
— 2-opt ;
— éventuellement des opérateurs inter-route plus complexes.

7.2. Première implémentation des voisinages

Nous avons commencé par implémenter les voisinages les plus classiques, car ils offrent un bon compromis entre simplicité de codage et capacité d'amélioration des solutions.

— Le voisinage `relocate` permet de réaffecter un client à une autre position, ce qui est particulièrement utile pour rééquilibrer les tournées.
— Le voisinage `swap` permet d'échanger deux clients et peut corriger rapidement de mauvaises affectations.
— Le voisinage `2-opt` agit à l'intérieur d'une tournée en supprimant certains croisements et en raccourcissant localement les trajets.

Sur une solution initiale aléatoire de l'instance `data101.vrp`, la combinaison de ces trois voisinages a permis de générer plusieurs milliers de voisins faisables et d'identifier immédiatement une solution améliorée. Ce résultat confirme que ces opérateurs constituent une base pertinente pour les métaheuristiques à venir.

7.3. Première implémentation du recuit simulé

Le recuit simulé a été implémenté dans une première version pragmatique. L'idée est de partir d'une solution initiale faisable, puis de répéter les étapes suivantes :

1. tirer un voisin faisable de manière aléatoire ;
2. calculer la variation de coût associée ;
3. accepter systématiquement un voisin meilleur ;
4. accepter parfois un voisin moins bon avec une probabilité dépendant de la température ;
5. diminuer progressivement la température ;
6. conserver la meilleure solution rencontrée pendant toute l'exécution.

Nous avons volontairement choisi, pour cette première version, de générer un voisin aléatoire faisable plutôt que d'énumérer tous les voisins à chaque itération. Ce choix réduit fortement le coût d'une itération et rend l'algorithme plus adapté aux tests répétés sur des instances de taille 100.

7.4. Résultats préliminaires

Les premiers essais montrent déjà que le recuit simulé améliore nettement les solutions initiales aléatoires. Par exemple, sur l'instance `data101.vrp`, un test de validation a permis de passer d'une distance initiale de 3488.41 à une meilleure distance de 2211.87 en 500 itérations, tout en conservant une solution faisable.

Ces résultats restent préliminaires. Ils servent à valider l'implémentation et à préparer les futurs tests expérimentaux plus longs, qui feront l'objet d'un protocole plus rigoureux dans la suite du projet.

7.5. Première implémentation de la recherche tabou

La recherche tabou a également été implémentée dans une première version opérationnelle. À chaque itération, l'algorithme génère un ensemble de voisins faisables, puis sélectionne le meilleur voisin autorisé, c'est-à-dire non tabou, sauf si un critère d'aspiration permet d'accepter une solution améliorant le meilleur résultat global.

Dans cette première version, la mémoire tabou porte sur les signatures des solutions récemment visitées. Cette approche n'est pas la plus fine possible, mais elle permet de disposer rapidement d'une base fiable pour les expériences comparatives.

7.6. Observation importante sur le coût de calcul

Les premiers essais montrent que la recherche tabou est sensiblement plus coûteuse en temps de calcul que le recuit simulé dans notre implémentation actuelle. Cette différence s'explique par le fait que, même avec une limitation du nombre de voisins inspectés, la recherche tabou évalue un ensemble de candidats à chaque itération alors que le recuit simulé peut se contenter d'un voisin aléatoire faisable.

Cette observation est importante pour la suite du projet. Elle signifie que les tests de validation rapide et les tests expérimentaux longs devront être distingués clairement, avec des paramètres adaptés à chaque usage.

7.7. Résultats préliminaires de la recherche tabou

Sur l'instance `data101.vrp`, un test ciblé a permis d'améliorer une solution initiale de distance 3604.26 vers une distance 3278.86 en 10 itérations. Un aperçu plus léger intégré au programme principal améliore également la solution initiale, mais avec un coût de calcul déjà notable même pour un très petit nombre d'itérations.

Ces résultats confirment que la méthode fonctionne, tout en montrant qu'une attention particulière devra être portée au réglage des paramètres et au protocole de test.

7.8. Mise en place du protocole expérimental

Afin d'éviter de mélanger validation fonctionnelle et comparaison scientifique, nous avons centralisé les expériences dans un module dédié. Ce module permet :

- de fixer des jeux de paramètres reproductibles ;
- de lancer plusieurs répétitions avec des graines différentes ;
- de résumer les résultats sous forme de moyennes, minimums et maximums ;
- de comparer directement les distances finales et les temps de calcul des deux métaheuristiques.

Dans l'état actuel, deux préréglages sont utilisés :

- un préréglage `quick`, pour les vérifications rapides ;
- un préréglage `long`, destiné aux futurs tests approfondis.

Les premières tentatives ont toutefois montré que le réglage initial du mode `long` était trop ambitieux pour la recherche tabou dans l'état actuel de l'implémentation. Nous avons donc recalibré ce préréglage afin d'obtenir un compromis plus réaliste entre profondeur de recherche et temps de calcul.

Afin de lancer ces expériences de manière explicite, nous avons également ajouté un mode dédié dans le programme principal. Cela permet de distinguer clairement :

- l'exécution courante du projet, utilisée pour les vérifications rapides ;
- les campagnes expérimentales ciblées, lancées volontairement avec un préréglage et un sous-ensemble d'instances.

Par exemple, une campagne ciblée sur une seule instance peut être lancée avec une commande du type :

```
python3 src/main.py --mode experiment --preset quick --instances data101.vrp --details
```

Cette commande produit à la fois un tableau récapitulatif et un détail par répétition, ce qui facilite la consignation des résultats dans le rapport.

7.9. Premiers résultats comparatifs

Le protocole `quick` a déjà été validé sur l'instance `data101.vrp`. Sur ce premier essai, la distance moyenne initiale observée est 3726.26. Le recuit simulé ramène cette valeur à 2295.29 en moyenne, alors que la recherche tabou atteint 3592.33 avec les paramètres rapides actuellement retenus.

Le temps de calcul moyen mesuré dans ce mode rapide est d'environ 0.44 seconde pour le recuit simulé contre environ 20.51 secondes pour la recherche tabou dans notre environnement final de relance. Cette différence confirme que, dans notre implémentation actuelle, la recherche tabou est nettement plus coûteuse et devra être utilisée avec précaution lors des campagnes expérimentales longues.

Le sujet demande aussi une comparaison en nombre de solutions générées. Pour quantifier ce point, nous avons ajouté un suivi explicite du nombre de voisins faisables réellement traités par chaque méthode. Sur `data101.vrp` avec le préréglage `quick`, le recuit simulé génère en moyenne 300 voisins faisables et en accepte 141, alors que la recherche tabou explore 200 voisins et n'accepte que 5 déplacements.

Cette différence s'explique directement par les paramètres retenus et par la logique des deux algorithmes :

— le recuit simulé génère un voisin faisable par itération, ici 300 itérations ;
— la recherche tabou inspecte à chaque itération un ensemble borné de voisins, ici jusqu'à 40 voisins pendant 5 itérations ;
— les deux méthodes utilisent les mêmes structures de voisinage (`relocate`, `swap`, `2-opt`), mais leur manière de les exploiter est très différente.

Ce point complète utilement la comparaison temps/qualité : dans notre implémentation actuelle, la recherche tabou paie son coût en temps par une exploration plus large et plus systématique des voisinages, tandis que le recuit simulé privilégie une progression légère mais très fréquente.

7.10. Premier essai en mode long

Un premier test en mode `long` a ensuite été lancé sur l'instance `data101.vrp`, avec deux répétitions. La distance moyenne initiale observée est 3719.84. Dans ce cadre plus exigeant, le recuit simulé obtient une distance moyenne de 1814.38, alors que la recherche tabou recalibrée atteint 2419.86.

Les temps moyens associés à cette expérience sont d'environ 0.56 seconde pour le recuit simulé et 1.01 seconde pour la recherche tabou.

Ce premier résultat long confirme deux points importants :

— le recuit simulé améliore fortement les solutions initiales avec un coût de calcul encore modéré ;
— la recherche tabou devient nettement plus compétitive qu'auparavant, mais reste derrière le recuit simulé sur cette première instance capacitaire.

Ces observations guideront la suite de notre protocole expérimental, notamment pour le choix des instances à tester en mode long et pour le réglage plus fin des paramètres de la recherche tabou.

7.11. Extension du test long à `data102.vrp`

Nous avons ensuite appliqué le même protocole `long` à l'instance `data102.vrp`. La distance moyenne initiale obtenue est 3609.70. Le recuit simulé atteint alors une distance moyenne de 1765.58, tandis que la recherche tabou obtient 2389.28.

Les temps moyens observés sont du même ordre que pour `data101.vrp` : environ 0.50 seconde pour le recuit simulé contre environ 0.88 seconde pour la recherche tabou.

Ce second résultat renforce la tendance observée sur la première instance : dans notre implémentation actuelle et avec les paramètres retenus, le recuit simulé offre un compromis nettement plus favorable entre qualité de solution et temps de calcul que la recherche tabou.

7.12. Résultats sur `data1101.vrp` et `data1102.vrp`

Nous avons ensuite appliqué le même protocole `long` aux instances `data1101.vrp` et `data1102.vrp`, qui présentent une demande totale plus élevée et nécessitent 9 véhicules dans la version sans fenêtres de temps.

Pour `data1101.vrp`, la distance moyenne initiale observée est 4801.67. Le recuit simulé obtient une distance moyenne de 2322.33, tandis que la recherche tabou atteint 3202.56.

Pour `data1102.vrp`, la distance moyenne initiale est 4702.59. Le recuit simulé descend à 2455.44 en moyenne, alors que la recherche tabou obtient 3063.83.

Les temps moyens restent faibles sur ces deux instances : environ 1.19 et 1.71 seconde pour le recuit simulé, contre environ 1.90 et 2.75 secondes pour la recherche tabou.

Ces nouveaux résultats confirment la tendance déjà observée sur `data101.vrp` et `data102.vrp`. Même sur des instances plus chargées, le recuit simulé améliore très fortement les solutions initiales pour un coût de calcul modéré, alors que la recherche tabou demeure beaucoup plus coûteuse tout en restant, dans notre implémentation actuelle, moins performante en qualité de solution finale.

7.13. Résultats sur `data111.vrp` et `data112.vrp`

Le même protocole `long` a ensuite été appliqué aux instances `data111.vrp` et `data112.vrp`. Les résultats observés confirment les mêmes tendances favorables au recuit simulé, même si les valeurs ne coïncident plus exactement avec celles de `data101.vrp` et `data102.vrp` dans notre relance finale.

On observe ici :

- `data111.vrp` : initiale 3480.20, recuit 1700.26, tabou 2306.55 ;
- `data112.vrp` : initiale 3550.56, recuit 1738.23, tabou 2317.44.

Les temps mesurés restent modestes pour les deux méthodes, autour de 0.56 à 0.78 seconde pour le recuit simulé et de 1.08 à 1.26 seconde pour la recherche tabou. Ces deux instances renforcent donc la cohérence de la tendance globale en faveur du recuit dans la version capacitaire.

7.14. Résultats sur `data1201.vrp` et `data1202.vrp`

Nous avons ensuite testé le protocole `long` sur les instances `data1201.vrp` et `data1202.vrp`, qui se distinguent par une capacité beaucoup plus élevée et un nombre de véhicules minimal égal à 2.

Pour `data1201.vrp`, la distance moyenne initiale observée est 4584.95. Le recuit simulé atteint une distance moyenne de 2001.83, tandis que la recherche tabou obtient 2905.82.

Pour `data1202.vrp`, la distance moyenne initiale vaut 4458.85. Le recuit simulé descend à 2009.33 en moyenne, alors que la recherche tabou atteint 2844.22.

Le point notable ici est que les deux méthodes deviennent très rapides : environ 0.43 à 0.51 seconde pour le recuit simulé et environ 0.65 à 0.73 seconde pour la recherche tabou.

Ces résultats renforcent encore l'avantage pratique du recuit simulé dans notre implémentation actuelle : même lorsque la structure des solutions change fortement, il reste à la fois efficace en qualité et très peu coûteux en temps de calcul.

7.15. Résultats sur `data201.vrp` et `data202.vrp`

Enfin, nous avons appliqué le protocole `long` aux instances `data201.vrp` et `data202.vrp`, qui utilisent elles aussi 2 véhicules dans la version sans fenêtres de temps.

Pour `data201.vrp`, la distance moyenne initiale observée est 3324.19. Le recuit simulé obtient une distance moyenne de 1669.67, tandis que la recherche tabou atteint 2147.25.

Pour `data202.vrp`, la distance moyenne initiale vaut 3401.22. Le recuit simulé descend à 1611.37 en moyenne, alors que la recherche tabou obtient 2219.52.

Comme pour `data1201.vrp` et `data1202.vrp`, les temps restent très faibles : environ 0.33 à 0.35 seconde pour le recuit simulé et autour de 0.52 à 0.53 seconde pour la recherche tabou.

7.16. Synthèse de la première campagne longue

À l'issue de cette première campagne longue menée sur l'ensemble des familles d'instances disponibles, une tendance forte se dégage :

- le recuit simulé améliore systématiquement et fortement les solutions initiales ;
- la recherche tabou améliore également nettement les solutions après recalibrage, mais reste globalement en retrait par rapport au recuit simulé dans la version capacitaire ;
- le coût de calcul du recuit simulé reste faible sur toutes les instances testées ;
- le coût de calcul de la recherche tabou devient lui aussi raisonnable, généralement entre 0.5 et 2.8 secondes par bloc expérimental.

Le tableau suivant résume les valeurs moyennes observées sur l'ensemble de cette première campagne longue.

Instance	k	Init. moy.	RS moy.	Tabou moy.	Temps RS	Temps tabou
data101.vrp	8	3719.84	1814.38	2419.86	0.56	1.01
data102.vrp	8	3609.70	1765.58	2389.28	0.50	0.88
data1101.vrp	9	4801.67	2322.33	3202.56	1.19	1.90
data1102.vrp	9	4702.59	2455.44	3063.83	1.71	2.75
data111.vrp	8	3480.20	1700.26	2306.55	0.78	1.26
data112.vrp	8	3550.56	1738.23	2317.44	0.56	1.08
data1201.vrp	2	4584.95	2001.83	2905.82	0.43	0.65
data1202.vrp	2	4458.85	2009.33	2844.22	0.51	0.73
data201.vrp	2	3324.19	1669.67	2147.25	0.35	0.52
data202.vrp	2	3401.22	1611.37	2219.52	0.33	0.53

Cette campagne améliorée fournit un signal plus net encore : dans la version capacitaire, le recuit simulé reste la meilleure méthode en qualité, tandis que la recherche tabou recalibrée devient beaucoup plus compétitive sans coûter significativement plus cher.

8. Format des données

Les fichiers d'entrée sont des fichiers texte au format `.vrp`. Une instance contient :

- le nom du fichier ;
- le nombre de dépôts ;
- le nombre de clients ;

— la capacité maximale d'un véhicule ;
— les informations sur le dépôt ;
— les informations sur chaque client.

Exemple de structure :

```
NAME: data101.vrp
COMMENT:
TYPE: vrptw
COORDINATES: cartesian
NB_DEPOTS: 1
NB_CLIENTS: 100
MAX_QUANTITY: 200

DATA_DEPOTS [idName x y readyTime dueTime]:
d1 35 35 0 230

DATA_CLIENTS [idName x y readyTime dueTime demand service]:
c1 41 49 161 171 10 10
...
```

Chaque client est donc décrit par :

— ses coordonnées (x, y) ;
— sa fenêtre de temps $[readyTime, dueTime]$;
— sa demande ;
— sa durée de service.

9. Bonus : résolution exacte sur sous-instances

Le sujet proposait en bonus d'utiliser un package de programmation linéaire afin d'identifier à partir de quelle taille d'instance il devient difficile d'obtenir une solution exacte. Nous avons traité ce point avec OR-Tools, via le solveur linéaire SCIP, sur la version capacitaire du problème.

L'idée retenue est simple : partir d'une instance existante, construire des sous-instances contenant seulement les n premiers clients, puis résoudre exactement chacune de ces sous-instances avec une limite de temps fixée. Nous avons automatisé cette étude dans un module dédié, de façon à pouvoir relancer facilement les tests sur plusieurs tailles.

Pour cette première étude, nous sommes partis de `data101.vrp` et avons testé les tailles 5, 10, 15, 20, 25, 30, 35 et 40 clients, avec une limite de 10 secondes par sous-instance.

Clients	k	Statut	Objectif	Temps (s)
5	1	OPTIMAL	119.48	0.03
10	1	OPTIMAL	173.04	0.01
15	2	OPTIMAL	248.91	0.74
20	2	OPTIMAL	280.13	0.46
25	2	OPTIMAL	335.27	5.08
30	3	FEASIBLE	364.88	10.01
35	3	FEASIBLE	444.35	10.01
40	3	FEASIBLE	561.89	10.01

L'enseignement principal est net : dans notre configuration, le solveur exact reste très efficace jusqu'à 25 clients, mais commence à peiner dès 30 clients, où il atteint déjà la limite de temps

sans certifier l'optimalité. Ce seuil ne constitue pas une frontière théorique universelle, mais il donne un ordre de grandeur concret sur le moment où une approche exacte devient sensiblement plus difficile à exploiter dans ce projet.

10. Limites et perspectives

Le travail réalisé permet de répondre à l'ensemble des points principaux du sujet, mais certaines limites doivent être signalées.

- Dans le cas VRPTW, la génération aléatoire de solutions initiales a été nettement améliorée, mais elle reste moins robuste que dans la version purement capacitaire.
- La recherche tabou recalibrée obtient de très bons résultats sur la campagne finale VRPTW, mais son coût de calcul demeure plus élevé sur quelques instances difficiles, en particulier lorsque le nombre de véhicules imposé par les fenêtres de temps devient important.
- L'étude exacte avec OR-Tools a volontairement été limitée à des sous-instances de petite taille ; elle fournit donc un ordre de grandeur utile, mais pas une frontière générale de difficulté.

Ces limites ouvrent des perspectives naturelles pour une poursuite du projet : renforcer encore l'initialisation aléatoire en VRPTW, affiner les paramètres ou la mémoire tabou sur les cas les plus contraints, et élargir l'étude exacte à d'autres familles de sous-instances.

11. Conclusion

Ce projet nous a permis de construire une chaîne complète de résolution pour le problème de tournées de véhicules, d'abord dans sa version capacitaire, puis dans sa version avec fenêtres de temps. Nous avons proposé une modélisation du problème, structuré le code en modules cohérents, développé des procédures de construction de solutions, puis implémenté et comparé deux métaheuristiques reposant sur les mêmes voisinages.

Les campagnes expérimentales finales dégagent une lecture claire. Dans le cas CVRP, le recuit simulé reste la méthode la plus efficace de notre étude, avec un excellent compromis entre qualité de solution et temps de calcul. Dans le cas VRPTW, les fenêtres de temps changent profondément la structure des solutions et augmentent nettement le nombre de véhicules nécessaires ; dans ce cadre plus contraint, la recherche tabou recalibrée devient globalement la meilleure méthode sur la campagne longue finale, même si elle conserve un surcoût sur certains cas difficiles.

Enfin, l'étude bonus menée avec OR-Tools confirme l'intérêt des approches heuristiques sur les instances de taille significative : dans notre cadre expérimental, la résolution exacte devient déjà sensiblement plus difficile à partir d'environ 30 clients. Au total, le projet aboutit donc à un résultat cohérent sur les plans algorithmique, expérimental et logiciel, tout en mettant en évidence des pistes d'amélioration réalistes pour une poursuite du travail.